1. System Software

1.1. Purposes of an Operating System (OS)

Maximizing Resource Use

- Memory
  - Partitioning main memory into static and dynamic partitions allowing for more than one program to be available.
  - Removing unused items from RAM as soon as the process is terminated and then marking the partition as available.
  - Use virtual memory with paging to swap parts of the memory and the disk.
- Disk
  - Compressing files in order to decrease the size and to allow for more larger files to be stored.
  - Using defragmentation software so that files are arranged contiguously in the disk thereby reducing access times.
  - Caching data that is frequently transferred to/from the disk either on the disk or in RAM.

User Interface

- Hides complexities of the hardware from the user
- gives users access to the hardware/software systems depending on their needs without involving them with the backend
- Examples include GUIs and CLIs

Multi-Tasking

- Managing the execution of multiple programs that appear to be running simultaneously.

Process

- A process is the code being executed.

Program

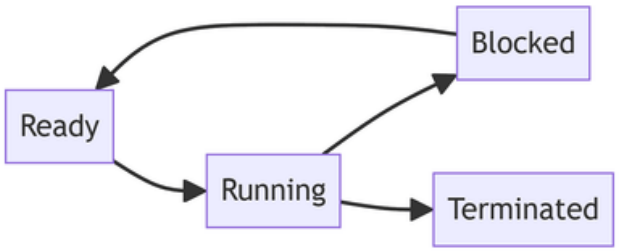
- A program is the code that has been written.

1.2. Process Scheduling

The process of allocating the CPU's time on different processes based on their needs.

Importance

- Allows for multi-tasking to take place
- Ensures the CPU is busy at all times
- Makes sure that all programs have fair access to the CPU so that they can be executed efficiently.
- Prevents starvation



Process States

- Ready: The process is not currently being executed. It is in the queue waiting for the processor's attention.
- Running: The process is being executed and currently has the processor's attention.
- Blocked: The process is waiting for an event before it can continue running like a user input.
- Terminated: The process has finished execution and has been removed from the queue.

Scheduling Algorithms

- Shortest Job First: Processes are executed in ascending order based on the time needed to execute the task with smaller jobs being executed first. This allows the CPU to execute more processes in a smaller amount of time. This is non-preemptive.
- Round Robin: Each process is allocated a specific amount of time from the CPU. Their execution takes place within these time frames. This helps prevent starvation.
- First Come First Served: No complex logic involved; processes are executed in the order they are requested. There is less processor overhead due to its simplicity.
- Shortest Remaining Time: Processes are executed in order of shortest remaining time for execution. It ensures there is smaller waiting times between process execution. This is preemptive.

Interrupt Handling

When an interrupt occurs, the process in the running state is moved to the ready state and the interrupt is moved to the running state so that it may be serviced.

Low Level Scheduler: Decides which of the processes in the ready state should be moved to the running state based on their priority/position.

High Level Scheduler: Decides which of the processes should be loaded from the backing store into the ready state.

1.3. Memory Management

Virtual Memory

Temporarily using secondary storage to simulate additional main memory, as only part of a program needs to be in RAM. Data is swapped between the two.

Paging

A large process is divided into pages of equal sizes.

1. The RAM is divided into frames.
2. Virtual memory is divided into same-sized blocks called pages with fixed sizes.
3. Page table created to translate logical address to physical addresses and track all the free frames.
4. Pages are swapped between them as necessary.
5. When there is no empty space in the memory, a page gets replaced with the new one. The replacement algorithm is either first in first out, least used page and recently least used page.

Page

Virtual memory is divided into equal blocks called pages.

Page Frame

Main memory is divided into parts equal in size to a page.

Page Table

Contains the mappings of pages to page frames.

Segmentation

A large process is divided to segments, the segments need not be the same size.

Disk Thrashing

When pages are needed in the RAM as soon as they are moved to the disk leading to a continuous swapping of the same pages because the pages being swapped in and out are inter-dependent.

Copyright © 2025 ZNotes Education & Foundation. All Rights Reserved.
This document is authorised for personal use only by at Enderun Colleges on 06/04/26.

1.4. Translation Software

Interpreter

- An interpreter reads line by line and checks statements for errors, if no errors are found it is executed otherwise a report is made and the interpreter stops execution. It is easier for debugging.
- A compiler does not run the code line by line but rather at the end of translation.

Lexical Analysis

- Characters are converted from strings into tokens.
- Comments are removed.
- Symbol table stores identifiers and keywords.
- White spaces are removed
- Keyword table stores the reserved words used and the matching operators.

Tokens: Strings with an assigned meaning

Syntax Analysis

A parse tree is generated for reverse polish notation and syntax errors are reported.

Code generation

Object code is generated.

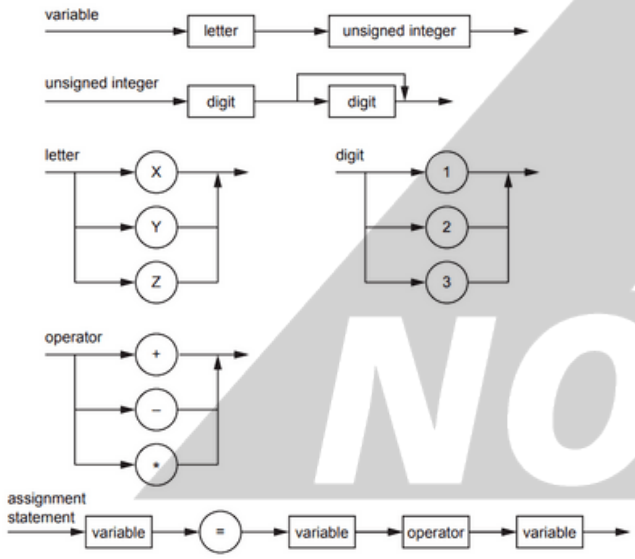
Code Optimization

Code is optimized to improve time for execution.

1.5. Backus-Naur Form (BNF) Notation

A form of expressing the grammar of a language visually.

Example




```
<variable> ::= <letter> <usigned integer>;

<usigned integer> ::= <digit> | <digit> <digit>;

<letter> ::= X | Y | Z;

<digit> ::= 1 | 2 | 3;

<operator> ::= + | - | *;

<assignment statement> ::== <variable> = <variable> <operator> <variable>;
```

Notations

< > : used to enclose an item
:: == : separates an item from its definition
| : between items, indicates a choice
; : the end of each rule.

1.6. Reverse Polish Notation (RPN)

Reverse Polish notation (RPN): An unambiguous method for representing an expression left to right without needing rules of precedence or brackets.

Advantages

- No brackets or precedence of operators needed
- Simpler to evaluate
- No need for backtracking in evaluation as the operators appear in the order required for computation and can be evaluated from left to right

Uses with a Stack

- RPN expression is read left to right
- As values are read numbers are pushed into the stack.
- If an operator is read the last two values in the stack are popped, the operation is performed on them, and the result is pushed back onto the stack.
- Repeated until the end of the expression is reached.

Example

$(a–b) * (a + c)/7 \quad \leftrightarrow \quad ab - ac + *7/$

$ab/4 * ab + - \quad \leftrightarrow \quad (a/b) * 4 - (a + b)$

Copyright © 2025 ZNotes Education & Foundation. All Rights Reserved.
This document is authorised for personal use only by at Enderun Colleges on 06/04/26.